

西南科技大学计算机科学与技术学院

实验报告

实 验 名 称 Linux 环境文件版本控制

实 验 地 点 东 11c225

实 验 日 期 2024 年 11 月 21 日

指 导 教 师 韦勇

学 生 班 级

学 生 姓 名

学 生 学 号

提 交 日 期 2024 年 11 月 25 日

一、实验目的

掌握源代码版本控制工具 git 的基本使用，理解源代码版本控制的原理；

二、实验设计

使用 git 工具完成从服务器上将源代码导出，将一个简单的测试项目的代码导出，并完成代码的修改和提交。

三、实验记录

任务 1：文件版本控制工具 git 的使用

请继续 7.2 的例子（如图 7 作家合作修改 README 的流程（主干方式）），将 README 文件修改为：

Chapter1

First

Second

Chapter2

First

其中，Chapter1 的部分由 A 完成，Chapter2 的部分由 B 完成。

1. 使用 git status 查看一下 A 工作目录的状态

```
[root@master A]# git status
# 位于分支 master
# 您的分支领先 'origin/master' 共 2 个提交。
# （使用 "git push" 来发布您的本地提交）
#
无文件要提交，干净的工作区
```

输出结果表明工作区没有任何未提交的更改。

2. 修改 README 文件内容为：

Chapter1

First

Second

```
[root@master A]# vim README
[root@master A]# cat README
Chapter1
First
Second
```

3. 修改完成后将 README 提交到远程代码库。

git add 用于添加文件到索引中。

git commit -m 用于提交到本地代码库中，-m 选项用于为这次提交设置一段描述文字，便于根据日志区别多次提交记录，可以通过 git log 查看提交日志。

git push 用于将本地代码库提交远程代码库中，可以通过"git remote -v"命令查看已有的远程代码库名称和对应的地址。

```
[root@master A]# git add README
[root@master A]# git commit -m "Create Chapter1 by A"
[master bf4094f] Create Chapter1 by A
1 file changed, 1 insertion(+), 2 deletions(-)
```

```
[root@master A]# git push origin master
Counting objects: 5, done.
Writing objects: 100% (3/3), 263 bytes | 0 bytes/s, done.
Total 3 (delta 0), reused 0 (delta 0)
To file:///root/workspace/rep.git
    e2f9517..bf4094f  master -> master
```

4. 切换到 B 工作目录，编辑章节 2 的内容

```
[root@master A]# cd ../B
[root@master B]# vim
[root@master B]# vim README
[root@master B]# cat README
Chapter2
First
```

5. 修改后使用 3 相同的命令提交到远程代码库

```
[root@master B]# git push origin master
To file:///root/workspace/rep.git
    ! [rejected]        master -> master (fetch first)
error: 无法推送一些引用到 'file:///root/workspace/rep.git'
提示：更新被拒绝，因为远程版本库包含您本地尚不存在的提交。这通常
是因为另外
提示：一个版本库已推送了相同的引用。再次推送前，您可能需要先合并
远程变更
提示：（如 'git pull'）。
提示：详见 'git push --help' 中的 'Note about fast-forwards' 小节。
```

这里报错，这是因为远程仓库有新的提交，而 B 的本地仓库没有这些提交的情况下。解决问题的方法通常是先拉取远程仓库的更新并合并，然后再尝试推送。

6. 拉取远程仓库

使用 `git pull origin master` 拉取仓库，出现提示信息：

冲突（内容冲突）：合并冲突于 README

自动合并失败，修正冲突然后提交修正的结果。

在合并过程中，README 文件出现了冲突，Git 无法自动解决。Git 提示合并失败，需要手动解决冲突，然后提交合并结果。

在 Git 中当两个分支尝试合并时，如果它们对同一文件的同一部分进行了不同的更改，就会发生冲突。Git 会标记这些冲突，以便可以手动解决它们。

在 README 中，<<<<<<< HEAD: 这一行标志着当前分支的更改开始。在<<<<<<< 和=====之间的内容是当前分支的更改。=====之后是另一个分支中 README 文件的内容。>>>>>>>后面的内容是另一个分支的提交 ID，用于标识这些更改来自哪个提交。

我们通过删除冲突标识来合并两部分内容。

```

[root@master B]# git pull origin master
remote: Counting objects: 5, done.
remote: Total 3 (delta 0), reused 0 (delta 0)
Unpacking objects: 100% (3/3), done.
来自 file:///root/workspace/rep
* branch          master      -> FETCH_HEAD
自动合并 README
冲突（内容）：合并冲突于 README
自动合并失败，修正冲突然后提交修正的结果。
[root@master B]# cat README
<<<<<< HEAD
Chapter2
=====
Chapter1
>>>>>> bf4094f05de0fb136d1ae9a039c9f04997e586ed
First
Second

```

7. 修改 README 文件，合并两个章节。

```

[root@master B]# vim README
[root@master B]# cat /root/workspace/B/README
Chapter1
First
Second
Chapter2
First

```

8. 再次尝试 push 本地仓库到远程仓库。成功

```

[root@master B]# git push origin master
Counting objects: 10, done.
Delta compression using up to 48 threads.
Compressing objects: 100% (2/2), done.
Writing objects: 100% (6/6), 514 bytes | 0 bytes/s, done.
Total 6 (delta 0), reused 0 (delta 0)
To file:///root/workspace/rep.git
    bf4094f..0a0d2b2  master -> master
[root@master B]# cat README
Chapter1
First
Second
Chapter2
First

```

四、实验思考或体会

1、以 7.2 中的过程为例子，描述源代码版本控制中的“冲突”是如何产生的？如何解决？

冲突产生的原因：

冲突产生于多个开发者对同一文件的同一部份进行不同的修改，并且尝试合并这些修改的时候，就会出现冲突。

在 7.2 中，A 和 B 都分别对 README 文件进行了修改，A 完成了 Chapter1 部分的修改并提交到远程代码库。B 在自己的工作目录修改 Chapter2 部分内容后，在没有更新本地仓库以包含 A 的提交的情况下就尝试将自己的修改提交到远程代码库。

由于远程代码库已经有了 A 提交的新内容，而 B 的本地仓库状态与之不同步，当 B 执行 `git push` 操作时就会报错。

当 B 执行 `git pull origin master` 命令来拉取远程仓库的更新时，Git 尝试自动合并 A 和 B 对 README 文件的修改。但由于他们修改的是同一个文件（README），并且修改的部分有重叠（虽然这里是不同章节，但在文件层面是同一文件），Git 无法自动确定如何合并这两种不同的修改，就产生了合并冲突。

当在合并过程中，文件出现了冲突，Git 会用特殊的标记来显示冲突部分来标识这些更改来自哪方，标记如下：

```
<<<<<<< HEAD 这一行标志着当前分支（B 所在分支）的更改开始
<<<<<<<和=====之间的内容是当前分支（B）的更改
=====之后是另一个分支（A）中 README 文件的内容
>>>>>>>后面的内容是另一个分支（A）的提交 ID
```

冲突解决方法：

先拉取更新并发现冲突，再打开冲突文件手动修改，最后再提交合并后的结果。

当出现冲突时，首先需要使用 `git pull origin master`（假设主分支是 `master`）命令拉取远程仓库的更新。这个操作会尝试自动合并远程仓库和本地仓库的内容，但如前面所述，当有冲突时会自动合并失败。

打开出现冲突的文件，找到 Git 标记的冲突部分，通过删除 `<<<<<<< HEAD`、`=====`、`>>>>>>>` 这些冲突标记，并合理地组合两个分支的修改内容来解决冲突。本次示例中就使用到了该方法。

在手动修改完成冲突文件后，再次提交这个合并后的结果。使用 `git add` 命令将修改后的文件添加到索引中，例如 `git add README`。

然后使用 `git commit -m` 命令提交到本地代码库中，`-m` 选项用于为这次提交设置一段描述文字，如 `git commit -m "Resolved merge conflict in README"`。

最后使用 `git push` 将本地代码库提交到远程代码库中，成功完成合并后的提交操作，使远程仓库中的 README 文件包含了 A 和 B 的修改内容